

Cómo personalizar el comportamiento de un modelo de IA

Depende del contexto, pero aquí van los escenarios más comunes:

1. Si usas una API (OpenAI, Anthropic, etc.)

Lo que modificas es el **system prompt** (mensaje de sistema):

Python

Ejemplo con OpenAI

```
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {
            "role": "system",
            "content": "Eres un asistente que solo responde en español y con tono formal. Nunca
            uses emojis."
        },
        {
            "role": "user",
            "content": "Hola, ¿qué tal?"
        }
    ]
)
```

No modificas archivos del modelo, sino los **parámetros de la petición**.

2. Si usas un framework local (Ollama, LM Studio, llama.cpp)

Archivos típicos a modificar:

text

proyecto/

- |—— Modelfile # (Ollama) Define comportamiento
- |—— config.json # Parámetros del modelo
- |—— system_prompt.txt # Tu prompt de sistema
- |—— generation_config.json # Temperatura, tokens, etc.

Ejemplo Modelfile (Ollama):

Dockerfile

FROM llama3

SYSTEM """

Eres un asistente experto en programación Python.

Siempre responde con ejemplos de código.

Nunca inventes información.

"""

PARAMETER temperature 0.7

PARAMETER top_p 0.9

PARAMETER num_ctx 4096

Ejemplo generation_config.json:

JSON

```
{
  "temperature": 0.7,
  "top_p": 0.9,
  "max_new_tokens": 2048,
  "repetition_penalty": 1.1
}
```

3. 🎯 Si quieres fine-tuning(entrenamiento personalizado)

Necesitas preparar estos archivos:

text

fine-tuning/

├── dataset.jsonl # Datos de entrenamiento

├── training_config.yaml # Configuración del entrenamiento

└── adapter_config.json # Si usas LoRA/QLoRA

Ejemplo dataset.jsonl:

JSON

```
{"messages": [{"role": "system", "content": "Eres un abogado"}, {"role": "user", "content": "¿Qué
```

```
es un contrato?"}, {"role": "assistant", "content": "Un contrato es un acuerdo..."}]]}
{"messages": [{"role": "system", "content": "Eres un abogado"}, {"role": "user", "content": "¿Qué
es una demanda?"}, {"role": "assistant", "content": "Una demanda es..."}]]}
```

Ejemplo training_config.yaml (con Hugging Face):

YAML

```
model_name: meta-llama/Llama-3-8B
learning_rate: 2e-5
num_epochs: 3
batch_size: 4
lora_r: 16
lora_alpha: 32
```

4. 🤖 Si usas LangChain / LlamaIndex

Python

```
# chain_config.py
from langchain.chat_models import ChatOpenAI
from langchain.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "Responde solo con datos verificables. Cita fuentes."),
    ("human", "{input}")
])

chain = prompt | ChatOpenAI(model="gpt-4", temperature=0.3)
```

Resumen rápido

Método	Qué modificas	Dificultad
System prompt	El mensaje de sistema	★ Fácil
Parámetros	temperature, top_p, etc.	★ Fácil
Modelfile	Archivo de configuración local	★★ Media

Fine-tuning	Dataset + config de entrenamiento	☆☆☆ Difícil
RAG	Base de conocimiento externa	☆☆ Media